

Guía Teórica de Repaso: Espacio Logarítmico (L y NL)

Definiciones, Propiedades, Reducciones y Teoremas Demostrados

Complejidad Computacional (FCEN — UBA)

Índice

1	Modelo de Cómputo para Espacio Sublineal	1
2	Configuraciones, Inclusiones y Jerarquía	2
2.1	Cota de Configuraciones para Espacio Sublineal	2
2.2	Inclusiones Fundamentales	2
3	Reducibilidad Logarítmica y NL-completitud	3
3.1	Insuficiencia de la Reducción Polinomial de Karp	3
3.2	Funciones Computables en Espacio Logarítmico	3
4	Teoremas para Demostrar en el Parcial	3
4.1	Teorema 1: PATH es NL-completo	3
4.2	Teorema 2: Caracterización de NL con Certificados Read-Once	4
4.3	Teorema 3: Teorema de Immerman-Szelepcsényi (NL = coNL)	5
5	Problemas Representativos en L y NL	6

1 Modelo de Cómputo para Espacio Sublineal

Cuando se estudian recursos de espacio sublineales ($S(n) < n$, como $S(n) = O(\log n)$), el modelo estándar de Máquina de Turing de una sola cinta resulta inadecuado, ya que la sola presencia de la entrada de tamaño n consumiría n celdas de memoria. Por esta razón, se adopta formalmente el **modelo de cinta separada (off-line Turing Machine)**.

Definición 1.1 (Modelo con Cintas Separadas). Una Máquina de Turing con espacio acotado se compone de:

- Cinta de entrada:** De *solo lectura (read-only)*. Los cabezales pueden moverse bidireccionalmente, pero no pueden escribir en ella. El espacio ocupado por esta cinta no cuenta para la cota de memoria.
- k cintas de trabajo:** De *lectura y escritura (read/write)* bidireccionales. El espacio utilizado por la máquina es el número total de celdas distintas visitadas en estas cintas a lo largo de todo el cómputo.
- Cinta de salida:** De *solo escritura (write-only)*. El cabezal se mueve exclusivamente a la derecha cada vez que escribe un símbolo. No cuenta para la cota de espacio.

Definición 1.2 (Clases L y NL).

$$\begin{aligned} \mathbf{L} &= \mathbf{SPACE}(\log n) = \bigcup_{c>0} \mathbf{SPACE}(c \log n) \\ \mathbf{NL} &= \mathbf{NSPACE}(\log n) = \bigcup_{c>0} \mathbf{NSPACE}(c \log n) \end{aligned}$$

L contiene los lenguajes decidibles por MT determinísticas en espacio de trabajo $O(\log n)$, y **NL** aquellos decidibles por MT no-determinísticas en espacio de trabajo $O(\log n)$.

Observación 1.3 (¿Por qué espacio logarítmico?). Una máquina con $c \log_2 n$ bits de memoria de trabajo puede mantener un número constante de **punteros** o **contadores** con valores hasta n , ya que cualquier índice entre 1 y n se codifica en binario con $\lceil \log_2 n \rceil$ bits. Esto permite referenciar posiciones arbitrarias de la cinta de entrada, contar pasos de algoritmos y almacenar identidades de vértices de un grafo.

2 Configuraciones, Inclusiones y Jerarquía

2.1 Cota de Configuraciones para Espacio Sublineal

Proposición 2.1 (Tamaño y Cantidad de Configuraciones). *Sea M una MT (determinística o no) que corre sobre una entrada x de longitud n utilizando a lo sumo $S(n)$ celdas de trabajo. Una configuración completa C almacena:*

- El estado interno actual $q \in Q$ ($O(1)$ bits).
- La posición del cabezal de entrada $i \in \{1, \dots, n\}$ ($\lceil \log_2 n \rceil$ bits).
- El contenido de las celdas utilizadas en las k cintas de trabajo ($S(n) \cdot \log |\Gamma|$ bits).
- Las posiciones de los cabezales en las cintas de trabajo ($k \cdot \lceil \log_2 S(n) \rceil$ bits).

Por lo tanto, la longitud en bits de C es:

$$|C| \leq O(1) + \log n + O(S(n))$$

Para $S(n) \geq \log n$, se tiene $|C| \leq c \cdot S(n)$. En consecuencia, el número total de configuraciones distintas posibles está acotado por:

$$|\text{Configs}(M, x)| \leq 2^{c \cdot S(n)} = 2^{O(S(n))}$$

Para espacio logarítmico ($S(n) = O(\log n)$):

$$|\text{Configs}(M, x)| \leq 2^{O(\log n)} = n^{O(1)} \quad (\text{un polinomio en } n)$$

2.2 Inclusiones Fundamentales

Teorema 2.2 (Inclusión $\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P}$). *Se cumple:*

$$\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P}$$

Demostración. La inclusión $\mathbf{L} \subseteq \mathbf{NL}$ es directa por definición, dado que toda MT determinística es un caso particular de MT no-determinística.

Para $\mathbf{NL} \subseteq \mathbf{P}$, sea $L \in \mathbf{NL}$ decidido por una MT no-determinística N en espacio $c \log n$. Construimos el grafo de configuraciones $G_{N,x} = (V, E)$ donde cada vértice es una configuración y existe una arista (C_1, C_2) si C_2 es alcanzable desde C_1 en un paso válido de transición de N .

- El número de vértices es $|V| \leq n^c$, por lo que el grafo tiene tamaño polinomial en n .
- Determinar si $x \in L$ equivale a resolver alcanzabilidad (*st-connectivity*) en $G_{N,x}$ desde la configuración inicial C_0 hacia alguna configuración aceptadora C_{acepta} .
- Como el problema de caminos en un grafo se resuelve en tiempo polinomial determinístico (por ejemplo, mediante BFS o DFS en tiempo $O(|V| + |E|) = O(n^{2c})$), se sigue que $L \in \mathbf{P}$.

□

Corolario 2.3 (Mapa de Inclusiones Globales).

$$\mathbf{NC}^1 \subseteq \mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXPTIME}$$

Por el Teorema de Jerarquía de Espacio se demuestra que $\mathbf{L} \subsetneq \mathbf{PSPACE}$, lo que garantiza que al menos una de las inclusiones intermedias debe ser estricta.

3 Reducibilidad Logarítmica y NL-completitud

3.1 Insuficiencia de la Reducción Polinomial de Karp

Proposición 3.1. Si utilizáramos la reducción de Karp ($\leq_p$), cualquier lenguaje no trivial $L \in \mathbf{NL}$ ($L \neq \emptyset$ y $L \neq \Sigma^*$) resultaría **NL-hard**.

Demostración. Como $\mathbf{NL} \subseteq \mathbf{P}$, para reducir cualquier $A \in \mathbf{NL}$ a L , una MT en tiempo polinomial simplemente decide si $x \in A$. Si $x \in A$, devuelve una cadena fija $y_{\text{sí}} \in L$; si $x \notin A$, devuelve $y_{\text{no}} \notin L$. $\square$

Por lo tanto, la reducción de Karp carece de poder discriminatorio dentro de $\mathbf{P}$. Se requiere una noción de reducción más restrictiva: la reducción en espacio logarítmico.

3.2 Funciones Computables en Espacio Logarítmico

Definición 3.2 (Función Computable Implícitamente en $\mathbf{L}$). Una función $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ es **computable implícitamente en $\mathbf{L}$** si satisface:

1. Crecimiento acotado polinomialmente: existe un polinomio p tal que $|f(x)| \leq p(|x|)$ para todo x .
2. El lenguaje $L_{\text{bit}} = \{\langle x, i \rangle : f(x)_i = 1\}$ pertenece a $\mathbf{L}$.
3. El lenguaje $L_{\text{len}} = \{\langle x, i \rangle : i \leq |f(x)|\}$ pertenece a $\mathbf{L}$.

Equivalentemente, existe una MT con cinta de salida *write-only* que escribe $f(x)$ usando a lo sumo $O(\log |x|)$ celdas en sus cintas de trabajo.

Definición 3.3 (Reducción $\leq_\ell$ y **NL-completitud**). • Decimos que $A \leq_\ell B$ si existe una función f computable en espacio logarítmico tal que $x \in A \iff f(x) \in B$.

- La reducción $\leq_\ell$ es **transitiva** y **cerrada hacia abajo**:

$$A \leq_\ell B \wedge B \in \mathbf{L} \implies A \in \mathbf{L}$$

- Un lenguaje B es **NL-completo** si $B \in \mathbf{NL}$ y para todo $A \in \mathbf{NL}$, $A \leq_\ell B$.

4 Teoremas para Demostrar en el Parcial

4.1 Teorema 1: PATH es NL-completo

Definición 4.1 (Problema PATH).

$$\text{PATH} = \{\langle G, s, t \rangle : G \text{ es un dígrafo y existe un camino dirigido de } s \text{ a } t\}$$

Teorema 4.2. *PATH es NL-completo bajo reducciones en espacio logarítmico $\leq_\ell$.*

Demostración. La demostración consta de dos partes: pertenencia a **NL** y dureza (**NL-hardness**).

1. Pertenencia ($\text{PATH} \in \mathbf{NL}$):

Diseñamos un algoritmo no-determinístico N que decide PATH en espacio $O(\log n)$, donde $n = |V|$:

1. Inicializar un registro de nodo actual $u \leftarrow s$ y un contador de pasos $k \leftarrow 0$.
2. Mientras $k < n$:
 - i. Si $u = t$, **aceptar**.
 - ii. Adivinar no-determinísticamente un vértice sucesor $v \in V$.
 - iii. Verificar si la arista $(u, v) \in E(G)$ leyendo la entrada. Si $(u, v) \notin E(G)$, la rama se descarta y **rechaza**.
 - iv. Actualizar $u \leftarrow v$ e incrementar $k \leftarrow k + 1$.
3. Si $k = n$ y $u \neq t$, **rechazar**.

Análisis de espacio: Almacenar los identificadores de los vértices u y v requiere $\lceil \log_2 n \rceil$ bits cada uno. El contador $k \leq n$ ocupa $\lceil \log_2 n \rceil$ bits. El espacio total de trabajo es $O(\log n)$. Por lo tanto, $\text{PATH} \in \text{NL}$.

2. Dureza ($\forall A \in \text{NL}, A \leq_\ell \text{PATH}$):

Sea $A \in \text{NL}$ un lenguaje decidido por una MT no-determinística M en espacio $c \log |x|$. Sin pérdida de generalidad, asumimos que M tiene una única configuración inicial C_0 y una única configuración de aceptación C_{acepta} , y que no efectúa pasos posteriores a la aceptación.

Fijada la entrada x con $n = |x|$, cada configuración de trabajo de M tiene longitud $m \leq d \log n$ para una constante d . El grafo de configuraciones $G_{M,x}$ posee a lo sumo $N = 2^{d \log n} = n^d$ vértices. Definimos la función de reducción:

$$f(x) = \langle G_{M,x}, C_0, C_{\text{acepta}} \rangle$$

Por definición de cómputo aceptador:

$$x \in A \iff \text{existe un camino dirigido de } C_0 \text{ a } C_{\text{acepta}} \text{ en } G_{M,x} \iff f(x) \in \text{PATH}$$

Computabilidad de f en espacio logarítmico: Para demostrar que f es implícitamente computable en L , se debe verificar que una MT determinística puede decidir si el par (C_p, C_q) es una arista en $G_{M,x}$ usando solo $O(\log n)$ espacio:

- Dadas las codificaciones binarias de dos configuraciones C_p y C_q (cada una de longitud $O(\log n)$), la máquina determinística simula la función de transición δ de M en un único paso a partir de C_p .
- Revisa si el estado, la celda leída de la entrada en la posición indicada por C_p y las celdas de trabajo permiten transicionar a C_q según alguna de las opciones no-determinísticas.
- Esta verificación requiere comparar cadenas de longitud $O(\log n)$, lo cual consume $O(\log n)$ espacio de trabajo.

Por ende, f es computable en espacio logarítmico y $A \leq_\ell \text{PATH}$. □

Corolario 4.3. $\text{L} = \text{NL} \iff \text{PATH} \in \text{L}$.

4.2 Teorema 2: Caracterización de NL con Certificados Read-Once

Teorema 4.4. *Un lenguaje $L \in \text{NL}$ si y solo si existen un polinomio p y una MT determinística V (verificador) que corre en espacio de trabajo $O(\log |x|)$, equipada con una cinta especial de certificado de lectura hacia adelante únicamente (read-once), tal que:*

$$x \in L \iff \exists u \in \{0, 1\}^{p(|x|)} \text{ tal que } V(x, u) = 1$$

Demostración. Dirección ($\Rightarrow$):

Sea $L \in \mathbf{NL}$ decidido por una MT no-determinística N en espacio $c \log |x|$. En cada paso, N tiene a lo sumo 2 ramas no-determinísticas. Como N se detiene en a lo sumo $T(n) = 2^{O(\log n)} = p(n)$ pasos, cualquier cómputo se puede codificar mediante una secuencia de elecciones binarias $u \in \{0, 1\}^{p(n)}$.

El verificador determinístico V simula a N sobre la entrada x . Cada vez que N debe tomar una decisión no-determinística, V lee el siguiente bit de la cinta del certificado u y avanza el cabezal hacia la derecha. V nunca necesita retroceder el cabezal de certificado ni almacenar elecciones previas; únicamente conserva la configuración actual de trabajo de N ($O(\log n)$ bits). Si N acepta, V acepta.

Dirección ($\Leftarrow$):

Sea V un verificador que utiliza espacio de trabajo $O(\log |x|)$ y lee u en modo *read-once*. Construimos una MT no-determinística N que simula a V . Cuando V solicita leer el siguiente bit del certificado u , N genera dicho bit de manera no-determinística sobre la marcha. Como V nunca retrocede su cabezal sobre u , N no requiere memorizar los bits adivinados en el pasado. El espacio de trabajo utilizado por N es idéntico al de V , es decir, $O(\log |x|)$. $\square$

Observación 4.5. Si se permitiera que el cabezal de la cinta del certificado se mueva hacia la izquierda (acceso bidireccional), el verificador podría releer el certificado tantas veces como deseara, lo cual definiría la clase $\mathbf{NP}$ y no $\mathbf{NL}$. La restricción *read-once* es la que preserva la cota de memoria logarítmica.

4.3 Teorema 3: Teorema de Immerman-Szelepcsényi ($\mathbf{NL} = \mathbf{coNL}$)

Definición 4.6 (Problema del Complemento de Conectividad).

$$\overline{\text{PATH}} = \{\langle G, s, t \rangle : G \text{ es un dígrafo y no existe camino dirigido de } s \text{ a } t\}$$

Teorema 4.7 (Immerman-Szelepcsényi, 1987–1988). $\mathbf{NL} = \mathbf{coNL}$.

Demostración. Dado que PATH es $\mathbf{NL}$ -completo bajo $\leq_\ell$, su complemento $\overline{\text{PATH}}$ es $\mathbf{coNL}$ -completo. Basta demostrar que $\overline{\text{PATH}} \in \mathbf{NL}$.

Sea $G = (V, E)$ un dígrafo con $n = |V|$ vértices y un nodo origen s . Definimos para cada $i \in \{0, 1, \dots, n\}$:

$$A_i = \{v \in V : \text{existe un camino en } G \text{ desde } s \text{ hasta } v \text{ de longitud a lo sumo } i\}$$

y denotamos $c_i = |A_i|$ a la cantidad de vértices alcanzables en hasta i pasos.

Estrategia de Conteo Inductivo (*Inductive Counting*):

Paso 1: Testigo de alcanzabilidad ($v \in A_i$).

Para verificar de forma no-determinística que $v \in A_i$, una máquina simplemente adivina un camino de longitud $\leq i$ desde s a v . Esto se realiza en espacio $O(\log n)$ guardando el vértice actual y un contador de pasos.

Paso 2: Testigo de no alcanzabilidad conociendo c_i ($v \notin A_i$).

Asumiendo que la máquina conoce el número exacto c_i , puede verificar no-determinísticamente que $v \notin A_i$ de la siguiente manera:

- a. Inicializar un contador $k \leftarrow 0$.
- b. Para cada vértice $u \in V$ en orden lexicográfico estricto $u_1 < u_2 < \dots < u_n$:
 - Adivinar no-determinísticamente si $u \in A_i$ o $u \notin A_i$.
 - Si adivina $u \in A_i$, verifica dicha afirmación encontrando un camino de s a u de longitud $\leq i$. Si el camino es válido, comprueba que $u \neq v$ (si $u = v$, la rama rechaza de inmediato) e incrementa $k \leftarrow k + 1$.
 - Si adivina $u \notin A_i$, no hace nada y continúa con el siguiente vértice.

- c. Al finalizar los n vértices, comprobar si $k = c_i$. Si $k = c_i$, la máquina concluye que efectivamente se certificaron todos los vértices de A_i y ninguno de ellos fue v ; por lo tanto, $v \notin A_i$. Si $k \neq c_i$, la rama de cómputo descarta la ejecución y rechaza.

Espacio del Paso 2: Solo se mantienen el índice u , el contador k y el camino de prueba actual, consumiendo $O(\log n)$ memoria.

Paso 3: Cálculo inductivo de la secuencia $c_0, c_1, \dots, c_n$.

- **Base:** $c_0 = 1$ (únicamente el conjunto $\{s\}$).
- **Paso inductivo (calcular c_{i+1} a partir de c_i):** Inicializar $c_{i+1} \leftarrow 0$. Para cada vértice $w \in V$:
 1. Adivinar no-determinísticamente si $w \in A_{i+1}$ o $w \notin A_{i+1}$.
 2. Si adivina $w \in A_{i+1}$, adivina un predecesor $u \in V$ tal que $(u, w) \in E$ (o $w = s$), y prueba que $u \in A_i$ mediante un camino de longitud $\leq i$. Si es exitoso, incrementa $c_{i+1} \leftarrow c_{i+1} + 1$.
 3. Si adivina $w \notin A_{i+1}$, debe verificar que para todo predecesor u con $(u, w) \in E$, se cumple $u \notin A_i$. Para ello, enumera todos los c_i elementos de A_i (utilizando el procedimiento del Paso 2) y verifica que ninguno de ellos sea adyacente hacia w .

Al terminar de recorrer los n vértices, se descarta el valor de c_i y se conserva únicamente c_{i+1} .

Conclusión del Algoritmo para $\overline{\text{PATH}}$:

Una vez obtenido c_n (el número total de vértices alcanzables desde s en cualquier cantidad de pasos), la máquina ejecuta el Paso 2 sobre el vértice de destino t verificando que $t \notin A_n$. Si la verificación es exitosa, la máquina **acepta**.

Como el algoritmo almacena únicamente un número constante de contadores y punteros de vértices en cada momento, el espacio total utilizado es $O(\log n)$. Por ende, $\overline{\text{PATH}} \in \text{NL}$, lo que concluye que $\text{NL} = \text{coNL}$. $\square$

Corolario 4.8 (Generalización para Espacios Mayores). *Para toda función construible en espacio $S(n) \geq \log n$:*

$$\text{NSPACE}(S(n)) = \text{coNSPACE}(S(n))$$

Observación 4.9 (Contraste Fundamental: Espacio vs. Tiempo). Mientras que en espacio el no-determinismo colapsa con su complemento ($\text{NL} = \text{coNL}$ y $\text{NSPACE} = \text{PSPACE} = \text{coPSPACE}$), en tiempo computacional la pregunta equivalente:

$$\text{NP} \stackrel{?}{=} \text{coNP}$$

permanece abierta y se conjetura ampliamente que $\text{NP} \neq \text{coNP}$.

5 Problemas Representativos en L y NL

Problema	Definición	Complejidad	Técnica / Algoritmo
EVEN	Cantidad par de unos en $x \in \{0, 1\}^*$	L	Un bit de memoria de trabajo que invierte con cada bit '1'.
PALINDROME	Cadenas capicúa: $w = w^R$	L	Dos contadores de índices i, j de $O(\log n)$ bits.
BALANCED	Igual cantidad de 0s y 1s en x	L	Contador binario de balance acotado por n .
PATH	Conectividad dirigida de s a t	NL -completo	Adivinar no-determinísticamente vértices sucesores.
2-SAT	Satisfacibilidad con cláusulas de tamaño 2	NL -completo	Grafo de implicaciones: $x \rightsquigarrow \neg x \rightsquigarrow x$ vía PATH.
STRONG-CONN	Grafo dirigido fuertemente conexo	NL -completo	Verificación de alcanzabilidad entre todos los pares.
BIPARTITE	Detección de grafos bipartitos	NL -completo	Equivalente a no poseer ciclos dirigidos impares.

Cuadro 1: Problemas canónicos en **L** y **NL**.